

Encoding Good Code as Typed Decisions

Jev, Scruple, and rule-driven quality for code written by LLMs



1. The problem

As developers depend more and more on LLMs to generate code for them, and as developers roles slip more into reviewing code than implementing code, many developers find themselves overwhelmed by the amount of code generated, which leads to mistakes in review processes. As well, LLMs code is subject to bloat or drift, in minor ways, that subtly lead to a mess of a codebase in a short time frame.

"Linting" has become industry standard by this point, but linters are unable to catch bloat and drift, because a linter acts on a line-by-line basis, and cannot understand meaning and wider context. Asking a traditional LLM to review an entire repository is unreliable, leading to hallucinations and missed context.

There are 2 new tools that I believe solve this problem with a new approach. **Scruple** breaks a codebase into small, bounded candidates and asks each one a single fixed question. **Jev**, a model from TypeSafe, answers each question with a typed decision and a confidence estimate rather than a conversation. Together they turn a written engineering ideology into rules that run like a linter but judge like a reviewer.

I created a Scruple plugin, enforcing some of my personal coding ideologies into Scruple Rules, [scruple-plugin-ideology](#), and I applied this against a codebase of a "todo" application written entirely by Claude Haiku. Using the rules, Haiku cleared most of the findings in a single pass. When it was also given examples of known-good code, it went beyond easy, local fixes to structural ones. The result suggests that much of code quality can come from the system around a model rather than from the model itself, which makes a small, inexpensive model a viable choice for writing production code, as well as increases the ability to trust the output of an LLM-driven coding process.

Existing tools don't close this gap:

- **Linters** work on syntax. They can tell you a variable is unused; they can't tell you whether a constructed object is a collaborator that should have been injected or a plain value that is fine to build inline.
- **LLM review of a whole repository** can reason about meaning, but it is unreliable in two ways. Given a large context, it misses entries that are right in front of it. And its output is free text, which differs from run to run and can't be used to pass or fail a build.

What teams need is a check that understands meaning but is as repeatable and as gateable as a linter.

2. Jev: decisions, not conversation

Most LLMs are built to drive a conversation. Jev is built to make a decision. It describes itself as a "system one" model: given one question with a fixed set of possible answers, it returns a typed result made of:

- the **choice** it made,

- a **probability** for each possible answer,
- an overall **confidence** estimate.

Because the result is typed, software can act on it directly: act when confidence is high, escalate to a human when it is not.

3. How a Scruple rule works

Scruple is an npm package that runs rules over a codebase. It does not put the codebase in front of a model and ask broad questions. Every rule follows the same pipeline:

```
collect()  code scan picks candidates (no model)
-> state    bounded excerpt: imports + function text (≤4000 chars)
-> question one fixed question
-> criteria  closed set of answers, incl. "can't tell"
-> diagnose  probability × confidence vs thresholds -> severity or nothing
```

1. **Collect.** A deterministic scan of the parsed code selects candidates. No model involvement at this step.
2. **State.** Each candidate is reduced to a bounded excerpt: the file's imports and the function's text (capped at 4,000 characters).
3. **Question and criteria.** The model is asked one fixed question and must choose from a closed set of answers.
4. **Diagnose.** The answer's probability and confidence are compared with thresholds to decide whether to report a finding, and at what severity.

Here is the question behind the `inject-dependencies` rule:

```
const instructions =
  'Does this function build a collaborator it then uses, instead of receiving it as a parameter?'

const criteria = {
  constructs_collaborator_internally: '...client, repository, service, connection... constructed here and used',
  constructs_plain_value:             '...built-in types, errors, dates, collections, DTOs, value objects...',
  returns_constructed_object:         '...the function is itself a factory.',
  insufficient_context:               '...imports and body do not establish what the constructed thing is.'
}
// ^ "can't tell" is an allowed answer

const defaultThreshold = { warning: 0.85, error: 0.95 }
const defaultMinConfidence = 0.7
```

Two design choices make this reliable:

- **False positives are named, not left to judgment.** In other words, a function who builds a `Date()` object or a custom `...OurDataShaped()` object, are named as expected outliers that should not trigger the rule.

- **"Can't tell" is an allowed answer.** When the excerpt doesn't contain enough information, the model has a sanctioned way to say so, and the rule stays silent instead of guessing.

The message attached to a finding is fixed text in the rule's source, so the same problem produces the same finding in every file and on every run.

4. Coverage and judgment are separate problems

A rule can fail in two distinct ways: a candidate is never examined, or it is examined and judged wrongly. Scruple and Jev address each one separately.

Approach	Never examined	Judged wrong
Whole repo → LLM	✗ misses entries	✗ free text or hallucinations
<code>collect()</code> → LLM	✓	⚠ no confidence to gate on
<code>collect()</code> → Jev	✓	✓ typed + confidence

- **Coverage** comes from Scruple's `collect()` step. Because candidates are selected by a deterministic scan and examined one at a time, nothing is skipped because a context was too large.
- **Judgment** comes from the typed decision. A general-purpose LLM placed behind the same `collect()` step would examine every candidate, but it could return prose with no calibrated confidence to set a threshold against. Even an LLM given structured output requirements and tooling to enforce that, can result in hallucinated outcomes.

5. Turning an engineering ideology into rules

Most teams have an engineering ideology, whether it is written down or not: inject dependencies, keep configuration in the environment, keep infrastructure at the edges, write comments only for the non-obvious *why*. Scruple gives teams the ability to turn their ideologies into enforceable rules.

6. Case study: `todo_test`

6.1 Setup

`todo_test` is a small Node.js todo application. Claude Haiku wrote the code shown in this section. The application was built in three stages:

1. **No rules:** Haiku built the application from feature tickets with no standards applied.
2. **Rules only:** Haiku was given Scruple's findings and asked to fix them.
3. **Rules plus known-good patterns:** Haiku was also given conventions as examples of what correct code looks like.

The first Scruple run over the whole repository reported 635 findings. After the first cleanup pass, 222 remained. Results for the individual rules in the rules-only stage:

Rule	Before → after
comments/no-useless-comments	418 → 3
ideology/no-swallowed-errors	38 → 3
ideology/no-repeated-literals	21 → 3
ideology/no-hidden-state	46 → 10

Three findings per rule were deliberately left in place as reference data. Of the 10 remaining `no-hidden-state` findings, 7 were introduced by the fix itself, a point we return to in Section 9.

6.2 notifications.js in three stages

The notifications module shows the difference between the three stages most clearly.

No rules — `src/notifications.js`

```
const NOTIFICATIONS_URL = process.env.NOTIFICATIONS_URL || 'http://localhost:4002'; // module-level config
const APP_ENV = process.env.APP_ENV || 'development';

function getNotifyApiKey() {
  ...
  return process.env.NOTIFY_API_KEY || 'test-key-123'; // hardcoded credential fallback
}

async function publishEvent(topic, payload) {
  try {
    await fetch(`${NOTIFICATIONS_URL}/topics/${topic}/publish`, { ... }); // no timeout
  } catch (error) {
    console.error(`Failed to publish event to topic ${topic}:`, error); // swallowed
  }
}
```

The unconstrained version reads configuration at module load, falls back to a hardcoded API key, makes outbound calls with no timeout, and logs errors and carries on.

Rules only

```
const SEND_ENDPOINT = '/send';
const JSON_CONTENT_TYPE = 'application/json';
const POST_METHOD = 'POST';           // literals extracted into constants
const PROVIDER_ERROR_STATUS = 500;
const MAX_ATTEMPTS = 3;

function getNotifyApiKey() {
  ...
  return process.env.NOTIFY_API_KEY || 'test-key-123'; // still here
}

} catch (error) {
  console.error(`Failed to publish event to topic ${topic}:`, error);
  throw error; // now rethrows
}
```

With rules only, Haiku fixed the easy, local findings: repeated literals became constants, and the swallowed error is now rethrown. The structural problems remained: the hardcoded credential fallback and the module-level configuration were untouched.

Rules plus patterns — src/api/notifications.js

```
class Notifications {
  #url; #apiKey; #enabled; #appEnv; #timeout; #maxAttempts
  constructor(config) { ... } // config injected, not read from env

  async publishEvent(topic, payload) {
    ...
    signal: AbortSignal.timeout(this.#timeout) // bounded outbound call
    ...
  }
}

function buildNotificationsConfig(env) { // env read once, at the edge
  ...
  let apiKey = null // no credential fallback
  ...
}
```

Given examples of the expected shape, Haiku made the structural fixes. Configuration is read once, at the edge of the application, and injected. The credential fallback is gone. Every outbound call is bounded by a timeout.

6.3 Clutter: no-useless-comments

Code written by LLMs tends to be heavily commented, and most of those comments restate the next line. In a single Haiku pass, `comments/no-useless-comments` went from 418 findings to 3. (again, 3 were left intentionally for reference)



Before — 9669603:src/notifications.js

```
// Notifications module for sending notifications and publishing events
// Send a notification with retry logic
async function sendNotification(to, subject, message) {
  // Retry on 500
  continue;
  // All retries failed, log and give up
  console.error('Failed to send notification after 3 attempts:', lastError);
  // Publish an event to a topic
  async function publishEvent(topic, payload) {
```

After — ecb4fa8

```
// Determine which API key to use (per-env override or fallback to NOTIFY_API_KEY)
function getNotifyApiKey() {
```

The comments that restated the code were removed. The comment that explains *why* the function exists was kept.

7. Small models, good outcomes

The case study suggests a loop in which a small model can produce code that meets a demanding standard:

```
Haiku writes code (guided by known-good shapes)
-> scruple: bounded candidates, typed answers
-> findings to Haiku
-> Haiku fixes, copying the shape
-> scruple re-runs (cached for unchanged code)
-> clean -> human review
```

Each part of the loop has one job:

- **Known-good shapes** (documentation, templates, exemplar code) show the model what correct code looks like.
- **Scruple rules** check every candidate against the standard, with a typed decision for each.
- **The model** applies the fixes, copying the shapes it has been shown.
- **Human review** happens once the code is clean.

When quality comes from the system rather than the model, choosing a model becomes a cost decision rather than a quality decision.

8. Adoption

The same mechanism works at two levels:

- **Organization level.** WWT can maintain security rule packs centrally and enforce them in every repository's CI pipeline.
- **Team level.** Teams can encode their own accepted patterns so that code written by LLMs cannot quietly break them.

Scruple is not tied to Jev. The decision provider is one line of configuration, and alternative providers can be plugged in, although few exist today.

9. Limits

Two limits matter in practice.

A rule only sees what its code scan selects. Coverage is guaranteed only for the candidates that `collect()` picks out. Anything outside the scan's patterns is never asked about. In `todo_test`, `no-hardcoded-config` did not flag this machine-specific path, because its patterns don't match file system paths:

```
src/api/index.js
```

```
app.use(express.static('/Users/wrigjame/code/todo_test/src/ui'));
```

Rules alone lead to the smallest change that clears the finding. A finding tells a model what is wrong, not what right looks like. In the rules-only stage, Haiku cleared repeated-literal findings by turning `'POST'` into `POST_METHOD`, and left the hardcoded credential in place (Section 6.2). Its fix for hidden state introduced 7 new findings. Known-good patterns are what turn minimal compliance into real improvement.

Conclusion

Asking an LLM to review a codebase gives you an opinion. Asking a typed-decision model one fixed question about each bounded piece of code gives you a measurement. Scruple and Jev make it possible to write an engineering ideology down once and enforce it on every change, at the scale at which LLMs now produce code.